

Dormitory Management System

Database Design Document



Version 1.0

September 2026

1. Introduction

1.1 Purpose

This document describes the database design for the Dormitory Management System. It serves as a comprehensive reference for developers, database administrators, and system maintainers.

1.2 Scope

This document covers:

- Database schema and structure
- Table definitions and relationships
- Data types and constraints
- Business rules and logic
- Indexing strategy

1.3 Database Management System

DBMS: MySQL 8.0+

Character Set: UTF-8

Collation: utf8mb4_unicode_ci

Storage Engine: InnoDB

2. Table Definitions

2.1 Users Table

Stores user accounts for all system users (owners, employees, and tenants).

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
name	VARCHAR(255)	NOT NULL	Full name
email	VARCHAR(255)	UNIQUE, NOT NULL	Email address (login credential)
phone	VARCHAR(20)	NULLABLE	Contact phone number
password	VARCHAR(255)	NOT NULL	Hashed password
is_active	BOOLEAN	DEFAULT TRUE	Account status
last_login_at	TIMESTAMP	NULLABLE	Last login timestamp
created_at	TIMESTAMP	NOT NULL	Account creation date
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

Business Rules:

- Email must be unique across all users
- Inactive users cannot login but records are preserved
- Passwords are hashed using bcrypt

2.2 Rooms Table

Stores dormitory room information and configuration.

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique room identifier
room_number	VARCHAR(10)	UNIQUE, NOT NULL	Room number (e.g., 101, 205)
floor	TINYINT UNSIGNED	NOT NULL	Floor number
capacity	TINYINT UNSIGNED	NOT NULL	Maximum number of tenants
monthly_rate	DECIMAL(10,2)	NOT NULL	Base monthly rent amount
is_airconditioned	BOOLEAN	DEFAULT FALSE	AC availability
meter_number	VARCHAR(30)	NULLABLE	Electricity meter identifier
status	VARCHAR(20)	DEFAULT 'vacant'	Room status: vacant, occupied, maintenance
created_at	TIMESTAMP	NOT NULL	Record creation date
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

Business Rules:

- Room number must be unique
- Capacity typically ranges from 1-4 tenants
- Status changes to 'occupied' when contract is created
- AC rooms typically have higher monthly rates

2.3 Tenants Table

Stores tenant information linked to user accounts.

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique tenant identifier
user_id	BIGINT UNSIGNED	FOREIGN KEY, UNIQUE, NOT NULL	Reference to users table
created_at	TIMESTAMP	NOT NULL	Record creation date
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

Relationship: One-to-One with Users table. Each tenant has exactly one user account.

2.4 Contracts Table

Stores rental agreements linking rooms with tenants.

Column	Type	Constraints	Description
<code>id</code>	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique contract identifier
<code>room_id</code>	BIGINT UNSIGNED	FOREIGN KEY, NOT NULL	Reference to rooms table
<code>start_date</code>	DATE	NOT NULL	Contract start date
<code>end_date</code>	DATE	NULLABLE	Contract end date (null = ongoing)
<code>due_day</code>	TINYINT UNSIGNED	DEFAULT 5	Day of month rent is due (1-28)
<code>deposit_required</code>	DECIMAL(10,2)	DEFAULT 0	Required security deposit amount
<code>deposit_collected</code>	DECIMAL(10,2)	DEFAULT 0	Amount of deposit collected
<code>status</code>	VARCHAR(20)	DEFAULT 'active'	Contract status: active, ended, ending_soon
<code>is_active</code>	BOOLEAN	DEFAULT TRUE	Active contract flag
<code>created_by</code>	BIGINT UNSIGNED	FOREIGN KEY, NULLABLE	User who created the contract
<code>created_at</code>	TIMESTAMP	NOT NULL	Contract creation date
<code>updated_at</code>	TIMESTAMP	NOT NULL	Last update timestamp

Business Rules:

- One room can have only one active contract at a time
- A room can have multiple contracts over time (historical)
- Due day must be between 1-28 to avoid month-end complications
- Deposit collected tracks total deposits from all tenants

2.5 Contract Tenants Table (Junction)

Many-to-many relationship between contracts and tenants.

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique identifier
contract_id	BIGINT UNSIGNED	FOREIGN KEY, NOT NULL	Reference to contracts
tenant_id	BIGINT UNSIGNED	FOREIGN KEY, NOT NULL	Reference to tenants
joined_on	DATE	NOT NULL	Date tenant joined contract
left_on	DATE	NULLABLE	Date tenant left (null = still active)
created_at	TIMESTAMP	NOT NULL	Record creation date
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

Business Rules:

- Multiple tenants can be assigned to one contract (room sharing)
- Tracks tenant move-in and move-out dates
- Number of active tenants should not exceed room capacity

2.6 Deposit Entries Table

Tracks all deposit transactions for transparency and accountability.

Column	Type	Constraints	Description
<code>id</code>	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique entry identifier
<code>contract_id</code>	BIGINT UNSIGNED	FOREIGN KEY, NOT NULL	Reference to contracts
<code>type</code>	VARCHAR(20)	NOT NULL	Transaction type: collected, deduction, refund
<code>amount</code>	DECIMAL(10,2)	NOT NULL	Transaction amount
<code>date</code>	DATE	NOT NULL	Transaction date
<code>reason</code>	VARCHAR(255)	NULLABLE	Reason for transaction
<code>recorded_by</code>	BIGINT UNSIGNED	FOREIGN KEY, NULLABLE	User who recorded the transaction
<code>created_at</code>	TIMESTAMP	NOT NULL	Record creation date
<code>updated_at</code>	TIMESTAMP	NOT NULL	Last update timestamp

Business Rules:

- **collected:** Initial deposit or additional deposits received
- **deduction:** Amount deducted for damages or unpaid rent
- **refund:** Amount returned to tenant at move-out
- Deductions require detailed reason for transparency
- Current balance = SUM(collected) - SUM(deduction) - SUM(refund)

2.7 Invoices Table

Stores monthly billing invoices for each contract.

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique invoice identifier
invoice_number	VARCHAR(30)	UNIQUE, NOT NULL	Invoice number (e.g., INV-2609-101)
contract_id	BIGINT UNSIGNED	FOREIGN KEY, NOT NULL	Reference to contracts
billing_month	VARCHAR(7)	NOT NULL	Billing period (YYYY-MM format)
due_date	DATE	NOT NULL	Payment due date
tenant_count	TINYINT UNSIGNED	NOT NULL	Number of tenants sharing the bill
rent_amount	DECIMAL(10,2)	NOT NULL	Room rent for the month
electricity_amount	DECIMAL(10,2)	DEFAULT 0	Electricity charges
carry_over_balance	DECIMAL(10,2)	DEFAULT 0	Unpaid balance from previous invoices
credit_balance	DECIMAL(10,2)	DEFAULT 0	Overpayment credit from previous invoices
total_amount	DECIMAL(10,2)	NOT NULL	Total amount due (rent + electricity + carryover - credit)
amount_paid	DECIMAL(10,2)	DEFAULT 0	Total amount paid so far
status	VARCHAR(20)	DEFAULT 'pending'	Invoice status: pending, paid, partial, late, overdue, void
created_by	BIGINT UNSIGNED	FOREIGN KEY, NULLABLE	User who created the invoice
created_at	TIMESTAMP	NOT NULL	Invoice creation date
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

Business Rules:

- One invoice per contract per month
- Status changes based on payment completion and due date
- Carry-over balances from unpaid previous invoices
- Credit balances from overpayments applied to current invoice
- Per-tenant share = $\text{total_amount} \div \text{tenant_count}$

2.8 Tenant Payments Table

Tracks individual tenant payment responsibility for shared invoices.

Column	Type	Constraints	Description
<code>id</code>	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique identifier
<code>invoice_id</code>	BIGINT UNSIGNED	FOREIGN KEY, NOT NULL	Reference to invoices
<code>tenant_id</code>	BIGINT UNSIGNED	FOREIGN KEY, NOT NULL	Reference to tenants
<code>share_amount</code>	DECIMAL(10,2)	NOT NULL	Tenant's share of total invoice
<code>amount_paid</code>	DECIMAL(10,2)	DEFAULT 0	Amount paid by this tenant
<code>status</code>	VARCHAR(20)	DEFAULT 'pending'	Payment status: pending, paid, late, overdue
<code>method</code>	VARCHAR(30)	NULLABLE	Payment method used
<code>paid_at</code>	TIMESTAMP	NULLABLE	Payment completion timestamp
<code>created_at</code>	TIMESTAMP	NOT NULL	Record creation date
<code>updated_at</code>	TIMESTAMP	NOT NULL	Last update timestamp

Business Rules:

- One row per tenant per invoice
- $\text{Share amount} = \text{invoice total_amount} \div \text{tenant_count}$
- Unique constraint on (invoice_id, tenant_id)
- Invoice status updates based on all tenant payment statuses

2.9 Payments Table

Records individual payment transactions.

Column	Type	Constraints	Description
<code>id</code>	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique payment identifier
<code>invoice_id</code>	BIGINT UNSIGNED	FOREIGN KEY, NOT NULL	Reference to invoices
<code>tenant_payment_id</code>	BIGINT UNSIGNED	FOREIGN KEY, NULLABLE	Reference to tenant_payments
<code>amount</code>	DECIMAL(10,2)	NOT NULL	Payment amount
<code>method</code>	VARCHAR(30)	NOT NULL	Payment method: cash, gcash, maya, bank_transfer, card
<code>received_at</code>	DATE	NOT NULL	Date payment was received
<code>reference</code>	VARCHAR(100)	NULLABLE	Transaction reference number
<code>notes</code>	TEXT	NULLABLE	Additional notes
<code>recorded_by_user</code>	BIGINT UNSIGNED	FOREIGN KEY, NULLABLE	User who recorded the payment
<code>created_at</code>	TIMESTAMP	NOT NULL	Record creation date
<code>updated_at</code>	TIMESTAMP	NOT NULL	Last update timestamp

Business Rules:

- Multiple payments can be made for one invoice
- Partial payments are supported
- Each payment updates the related tenant_payment and invoice records

2.10 Meter Readings Table

Stores monthly electricity meter readings for billing calculations.

Column	Type	Constraints	Description
<code>id</code>	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique identifier
<code>room_id</code>	BIGINT UNSIGNED	FOREIGN KEY, NOT NULL	Reference to rooms
<code>reading_month</code>	VARCHAR(7)	NOT NULL	Reading period (YYYY-MM)
<code>previous_kwh</code>	DECIMAL(10,2)	NOT NULL	Previous month's meter reading
<code>current_kwh</code>	DECIMAL(10,2)	NOT NULL	Current meter reading
<code>kwh_used</code>	DECIMAL(10,2)	NOT NULL	Consumption (current - previous)
<code>recorded_by</code>	BIGINT UNSIGNED	FOREIGN KEY, NULLABLE	User who recorded the reading
<code>created_at</code>	TIMESTAMP	NOT NULL	Record creation date
<code>updated_at</code>	TIMESTAMP	NOT NULL	Last update timestamp

Business Rules:

- One reading per room per month
- `kwh_used` is calculated automatically: `current_kwh - previous_kwh`
- Electricity charges = `kwh_used` × `electricity_rate` (from settings)
- Unique constraint on (`room_id`, `reading_month`)

2.11 Services Table (Laundry)

Defines available laundry services.

Column	Type	Constraints	Description
<code>id</code>	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique service identifier
<code>code</code>	VARCHAR(20)	UNIQUE, NOT NULL	Service code (e.g., WASH, DRY, FOLD)
<code>name</code>	VARCHAR(100)	NOT NULL	Service name
<code>weight_limit_kg</code>	DECIMAL(5,2)	NOT NULL	Maximum weight per load
<code>is_active</code>	BOOLEAN	DEFAULT TRUE	Service availability
<code>created_at</code>	TIMESTAMP	NOT NULL	Record creation date
<code>updated_at</code>	TIMESTAMP	NOT NULL	Last update timestamp

2.12 Service Prices Table

Stores pricing for services by customer type.

Column	Type	Constraints	Description
<code>id</code>	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique identifier
<code>service_id</code>	BIGINT UNSIGNED	FOREIGN KEY, NOT NULL	Reference to services
<code>customer_type</code>	ENUM	NOT NULL	tenant, student, non_student
<code>price</code>	DECIMAL(8,2)	NOT NULL	Service price
<code>created_at</code>	TIMESTAMP	NOT NULL	Record creation date
<code>updated_at</code>	TIMESTAMP	NOT NULL	Last update timestamp

Business Rules:

- Unique constraint on (service_id, customer_type)
- Different pricing tiers for different customer types

2.13 Laundry Orders Table

Tracks laundry service orders from tenants.

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique order identifier
tenant_id	BIGINT UNSIGNED	FOREIGN KEY, NOT NULL	Reference to tenants
order_date	TIMESTAMP	NOT NULL	Order placement date/time
pickup_date	DATE	NOT NULL	Expected pickup date
status	VARCHAR(20)	DEFAULT 'pending'	pending, processing, ready, picked_up
notes	TEXT	NULLABLE	Special instructions
created_at	TIMESTAMP	NOT NULL	Record creation date
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

2.14 Order Items Table

Line items for laundry orders.

Column	Type	Constraints	Description
<code>id</code>	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique identifier
<code>laundry_order_id</code>	BIGINT UNSIGNED	FOREIGN KEY, NOT NULL	Reference to laundry_orders
<code>service_id</code>	BIGINT UNSIGNED	FOREIGN KEY, NOT NULL	Reference to services
<code>quantity</code>	INT	NOT NULL	Number of loads
<code>price</code>	DECIMAL(8,2)	NOT NULL	Price per unit at time of order
<code>subtotal</code>	DECIMAL(10,2)	NOT NULL	Line total (quantity × price)
<code>created_at</code>	TIMESTAMP	NOT NULL	Record creation date
<code>updated_at</code>	TIMESTAMP	NOT NULL	Last update timestamp

2.15 Settings Table

Stores system configuration values with version history.

Column	Type	Constraints	Description
<code>id</code>	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique identifier
<code>key</code>	VARCHAR(100)	NOT NULL	Setting key name
<code>value</code>	TEXT	NOT NULL	Setting value
<code>effective_from</code>	DATE	NULLABLE	Date setting takes effect
<code>set_by</code>	BIGINT UNSIGNED	FOREIGN KEY, NULLABLE	User who set the value
<code>created_at</code>	TIMESTAMP	NOT NULL	Record creation date
<code>updated_at</code>	TIMESTAMP	NOT NULL	Last update timestamp

Common Settings:

- **electricity_rate:** Cost per kWh with effective date
- **default_due_day:** Standard payment due day (1-28)
- **invoice_advance_days:** Days before month-end to generate invoices
- **overdue_grace_days:** Days after due date before marking overdue

2.16 Audit Logs Table

Comprehensive audit trail of all system activities.

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique log identifier
user_id	BIGINT UNSIGNED	FOREIGN KEY, NULLABLE	User who performed action
action	VARCHAR(50)	NOT NULL	Action type: Created, Updated, Deleted, etc.
model_type	VARCHAR(100)	NOT NULL	Entity type affected
model_id	BIGINT UNSIGNED	NULLABLE	Entity ID affected
description	TEXT	NOT NULL	Human-readable description
ip_address	VARCHAR(45)	NULLABLE	User IP address
created_at	TIMESTAMP	NOT NULL	Action timestamp

Business Rules:

- Immutable records (no updates or deletes)
- Complete trail of all significant system activities
- Used for compliance, troubleshooting, and dispute resolution

3. Entity Relationships

3.1 One-to-One Relationships

- **Users ↔ Tenants:** Each user can be one tenant, each tenant has one user account

3.2 One-to-Many Relationships

- **Rooms → Contracts:** One room can have multiple contracts over time
- **Contracts → Invoices:** One contract generates monthly invoices
- **Contracts → Deposit Entries:** One contract has multiple deposit transactions
- **Invoices → Tenant Payments:** One invoice split into payments per tenant
- **Invoices → Payments:** One invoice can have multiple payment transactions
- **Rooms → Meter Readings:** One room has monthly readings
- **Tenants → Laundry Orders:** One tenant can place multiple orders
- **Laundry Orders → Order Items:** One order contains multiple service items
- **Services → Service Prices:** One service has prices for different customer types
- **Users → Created Records:** One user creates multiple contracts, invoices, etc.

3.3 Many-to-Many Relationships

- **Contracts ↔ Tenants:** Multiple tenants can share one contract (junction: contract_tenants)
- **Laundry Orders ↔ Services:** One order can include multiple services (junction: order_items)

3.4 Referential Integrity

All foreign key relationships enforce cascading rules:

- **ON DELETE CASCADE:** When parent record is deleted, child records are also deleted (e.g., deleting a contract deletes its invoices)
- **ON DELETE SET NULL:** When parent record is deleted, foreign key is set to NULL (e.g., deleting a user sets created_by to NULL)

4. Indexing Strategy

4.1 Primary Keys

All tables use auto-incrementing BIGINT UNSIGNED primary keys for optimal performance.

4.2 Foreign Key Indexes

All foreign key columns are automatically indexed for join performance.

4.3 Unique Indexes

- `users.email` - Unique email addresses
- `rooms.room_number` - Unique room numbers
- `invoices.invoice_number` - Unique invoice numbers
- `services.code` - Unique service codes
- `(tenant_payments.invoice_id, tenant_payments.tenant_id)` - One payment per tenant per invoice
- `(service_prices.service_id, service_prices.customer_type)` - One price per service per customer type
- `(meter_readings.room_id, meter_readings.reading_month)` - One reading per room per month

4.4 Composite Indexes

Composite indexes for common query patterns:

- `(contracts.room_id, contracts.is_active)` - Find active contract for a room

- `(invoices.contract_id, invoices.billing_month)` - Find invoice for specific month
- `(invoices.status, invoices.due_date)` - Find unpaid invoices by due date

5. Business Logic & Calculations

5.1 Deposit Balance Calculation

```
current_balance = SUM(collected) - SUM(deduction) - SUM(refund)
```

Where entries are filtered by `contract_id` and grouped by `type` .

5.2 Invoice Total Calculation

```
total_amount = rent_amount + electricity_amount + carry_over_balance -  
credit_balance
```

5.3 Per-Tenant Share

```
share_amount = total_amount ÷ tenant_count
```

Automatically calculated when invoice is generated based on number of active tenants in contract.

5.4 Electricity Billing

```
electricity_amount = kwh_used × electricity_rate
```

Where `kwh_used = current_kwh - previous_kwh` from meter_readings.

5.5 Invoice Status Logic

- **Pending:** Not yet due, amount_paid = 0
- **Partial:** Some but not all tenants have paid
- **Paid:** All tenants have paid their full share

- **Late:** Past due date but within grace period
- **Overdue:** Past grace period, no payment
- **Void:** Cancelled (usually when contract is deactivated)

5.6 Carry-Over Balance

Unpaid balance from previous invoices is automatically added to the next invoice as

`carry_over_balance`.

5.7 Credit Balance

Overpayments are automatically applied as `credit_balance` to reduce the next invoice amount.